

2026년 개인주제탐구발표대회 보고서

지능형 큐브솔버 DLCM (Deep Learning Cube Machine)

2026. 9. 4.

과 정 명	AI
소속 학교	탄방중학교
성 명	박제욱

대전교육정보원정보영재교육원

I. 탐구의 필요성 및 목적

1. 탐구의 필요성

- 가. 작년 ‘SW고급’ 과정에서 Kociemba모듈을 활용하여 세계 상위권 선수의 기록을 깨기 위한 큐브 해법을 도출해 보았다. 이때에는 이미 개발되어 상용화된 모듈을 사용해서 해법을 도출해 냈지만, 이는 단지 기존의 알고리즘을 활용한 것에 불과하다.
- 나. 이번 연구에서는 기존 Kociemba모듈 사용이 아닌, 고도화된 알고리즘을 직접 개발할 필요성을 느꼈다. 이를 적용하기 위해 선택한 큐브 종목은 Fewest Moves Count(FMC)이다. FMC는 큐브를 최소 회전으로 맞추는 것을 기준으로 대결하는 종목으로 예전에 공식대회에서 이 종목에 도전한 적은 있다. 하지만 FMC에는 Normal-Inverse Scramble Switching(NISS), 블럭빌딩 등 습득해야 할 것이 너무 많아 당시에는 FMC의 다른 참가자들과 경쟁하는 것이 매우 어려웠다.
- 다. 이번 연구에서는 AI 머신러닝을 통해서 FMC 기록 단축을 위한 최소 회전 해법을 찾아보려고 한다.

2. 연구 목적

- 가. 큐브의 가장 효율적인 해법을 알고 싶은 사람들이 DLCM을 이용해서 간편하고 정확하게 해법을 알 수 있도록 하는 것이다.
- 나. 머신러닝을 이용해서 DLCM을 구축하고, 이를 Kociemba와 비교분석을 하고자 한다.
- 다. DLCM을 이용해서 FMC의 세계 기록보다 더 좋은 기록을 낼 수 있는지 확인하고자 한다.

II. 이론적 배경

1. 선행 연구

- 가. Kociemba 오픈소스 및 데이터 처리 방식 분석
 - 1) Kociemba는 Two-Phase Algorithm을 이용하여 큐브를 해결하며, 탐색 과정을 Phase 1과 Phase 2로 나누어 수행한다.
 - 2) 큐브의 상태를 각 면의 색상 정보가 포함된 문자열 형태로 입력 받아 내부적으로 큐브의 위치와 방향 등의 상태 정보로 변환하여 처리한다.
 - 3) 탐색 과정에서 미리 계산된 테이블과 가지치기를 활용하여 불필요한 탐색을 줄이고 해법을 탐색한다.

나. 딥러닝을 이용한 큐브 해법 연구 DeepCubeA

- 1) DeepCubeA는 딥러닝과 탐색을 결합하여 루빅스 큐브의 해법을 찾는 연구이다.
- 2) 큐브의 상태를 입력받아 해당 상태에서 Solved까지 필요한 비용을 예측하는 비용 함수를 학습한다.
- 3) 손실 함수를 이용해 탐색 알고리즘과 결합하여 해결 경로를 찾아낸다.

2. 용어 정의

가. 큐브 용어

- 1) 스크램블: 큐브를 섞기 위한 회전을 통칭하는 것이다.
- 2) 솔빙: 큐브를 맞추는 모든 과정이다.

나. 코딩 용어

- 1) 원-핫인코딩: 데이터를 0과 1로 구성된 벡터로 바꾸는 과정으로, 데이터의 오인식을 방지한다.
- 2) 신경망: 외부에서 데이터를 받아들이는 입력층, 입력층과 출력층 사이에서 데이터를 학습하는 은닉층, 모든 연산을 거쳐 최종 결과값을 만들어 내는 출력층으로 구성되어 있다. 각각의 은닉층에서는 가중치 및 편향으로 학습한다.
- 3) ResBlock: 입력값을 다음 층의 결과에 직접 더하는 구조를 이용하여 깊은 신경망의 학습을 돕는 구조이다.
- 4) 비용 함수: 현재 큐브 상태에서 완성 상태까지 필요한 비용을 예측하는 함수이다.
- 5) 벨만 타겟: 현재 상태에서 한번의 행동을 수행한 후 얻은 다음 상태의 예측값을 이용하여 현재 상태의 학습 목표값을 계산하는 방법이다.
- 6) A-star 알고리즘: 현재까지의 노드 n 까지 이동한 비용 $g(n)$, 현재 노드로부터 solved까지 남은 추정 거리 $h(n)$, 최종 예상 비용 $f(n)$ 에 대해 $f(n) = g(n) + h(n)$ 으로, $f(n)$ 이 작은 값을 먼저 탐색을 하는 알고리즘이다.
- 7) Batch 처리: 여러 개의 데이터를 한 번에 처리하는 방법이다.

Ⅲ. 탐구 방법 및 과정

1. 연구 방법

가. 자료 조사

- 1) Kociemba의 공식 홈페이지를 통해서 Kociemba의 오픈소스 및 데이터 처리 방식을 분석한다.
- 2) 딥러닝을 이용한 큐브 해법 연구 ‘DeepCubeA’의 논문을 논문 요약 시스템을 이용해 요약한 후, ‘DeepCubeA’의 딥러닝 방식을 조사한다.
- 3) 비용함수, 벨만타겟, A* 알고리즘 등의 원리를 조사한다.

나. 큐브 상태 구현

- 1) 3x3x3 루빅스 큐브의 6개의 면 색상을 0~5의 정수로 표현하여 큐브의 현재 상태를 배열로 저장한다.
 - RubiksCube 클래스에 저장, 완성상태: SOLVED_STATE
- 2) 큐브의 기본 회전인 U, U' , D, D' , R, R' , L, L' , F, F' , B, B' 의 총 12 가지 회전을 배열의 위치 변환으로 구현한다.
 - RubiksCube.apply_move() 에 저장
- 3) Solved 상태와 현재 상태를 비교하여 큐브의 해결 여부를 판단한다.

다. 데이터 처리 및 전처리

- 1) 여러 개의 큐브 상태를 동시에 처리할 수 있도록 Batch 방식의 회전 함수를 구현한다.
 - apply_move_batch()에 구현
- 2) 큐브의 각 색상 상태를 원-핫인코딩으로 색상을 6개의 값으로 표현한다. 이는 $54 \times 6 = 324$ 개로 표현된다.
 - to_one_hot_batch() 에 구현
- 3) 실제 탐색 과정에서도 큐브 상태를 동일한 방식으로 324 차원 데이터로 변환하여 모델에 입력한다.

라. 학습 데이터 생성

- 1) Solved 상태의 큐브에서 시작해서 무작위 회전을 통해 학습용 스크램블을 생성한다.
 - generate_scrambled_states()로 구현
- 2) 각 학습 데이터의 스크램블 길이를 10~38수 범위에서 무작위로 결정한다.

3) 직전에 수행한 회전의 반대 회전이 바로 선택되지 않도록 제한하여 불필요한 중복 상태의 생성을 줄인다.

4) 여러 개의 큐브 상태를 한 번에 생성하여 학습 데이터 생성 속도를 향상시킨다. (Batch 이용)

마. 벨만 타겟 생성

1) 하나의 큐브 상태에서 가능한 12개의 다음 상태를 생성한다.

2) Target Model(신경망)을 이용하여 각각의 다음 상태에서 Solved 까지 필요한 비용을 예측한다.

- target_model, child_values에 저장

3) 12개의 예측값 중 최솟값에 1을 더해 현재 상태의 벨만 타겟을 계산한다.

- $\text{child_values.min(axis=1)} + 1.0 = \text{target}$

바. 딥러닝 모델 설계

1) 324개의 입력값을 2000차원 은닉층으로 변환하는 입력층을 구성한다.

- DeepCubeNet 클래스, input_dim = 324, hidden_dim=2000

2) Batch Normalization과 ReLU 활성화 함수를 사용하여 신경망을 구성한다.

- nn.BatchNorm1d, nn.ReLU 이용

3) 여러층을 안정적으로 학습하기 위해 ResBlock 6개를 연결한다.

- num_res_blocks = 6

4) 마지막으로 현재 큐브 상태의 솔빙 비용을 하나의 실수 값으로 출력하도록 Value Head를 구성한다.

사. 딥러닝 모델 학습

1) 생성된 큐브 상태를 원-핫인코딩을 하여 신경망의 입력으로 변환한다.

2) 벨만 타겟과 신경망의 예측값 차이를 Smooth L1 Loss라는 손실 함수를 이용해서 계산한다.

- nn.SmoothL1Loss(beta=0.5)

3) Adam optimizer를 사용해 신경망 가중치를 업데이트한다.

4) 학습률을 변화시키기 위해 Cosine Annealing Warm Restarts를 사용한다. 이는 학습률을 조절하여 신경망의 결과가 정확하게 나오도록 도와준다.

- optim.lr_scheduler.CosineAnnealingWarmRestarts()

- 5) 변화량(기울기)가 지나치게 커지는 것을 방지하기 위해서 Gradient Clipping을 적용한다.

아. Target Model 및 체크포인트 관리

- 1) 학습 모델과 별도의 Target Model을 구성하여 벨만 타겟 계산에 사용한다.
- 2) 학습 도중 모델의 가중치, Target Model, Optimizer, Scheduler, 학습 횟수를 체크포인트로 저장한다.
- 3) 저장된 체크포인트중 가장 많이 학습된 모델을 자동으로 찾아 이어서 학습하도록 한다.

자. Weighted A-star을 이용한 해법 탐색

- 1) 학습된 신경망의 비용 함수를 A-star 탐색의 예상 비용으로 사용한다.
- 2) 현재까지 사용한 회전수와 예측 비용을 이용하여 탐색 우선순위를 계산한다.
- 3) 각 큐브 상태에서 가능한 다음 회전을 생성하고 신경망을 이용하여 각 상태의 비용을 계산한다.
- 4) 방문한 상태와 사용한 회전수를 저장하여 불필요한 탐색을 줄이고, 직전 회전의 반대 회전을 수행하지 않도록 제한한다.
- 5) 우선순위큐를 이용해서 평가값이 작은 상태부터 탐색하도록 구현한다.
- 6) 탐색 중 Solved 상태가 발견되면 해당 회전 순서를 해법으로 반환한다.

차. 실제 스�크램블 적용 및 성능 확인

- 1) FMC 공식 37수 스�크램블을 입력하여 실제 큐브 상태를 생성한다.
- 2) R2, D2등의 회전을 2개의 회전으로 쪼갬다.
 - expanded_scramble으로 변환
- 3) 스�크램블을 RubiksCube.apply_move()에 순차적으로 적용하여 테스트용 큐브 상태를 생성한다.
- 4) 학습된 DLCM에 해당 큐브 상태를 입력하여 해법을 탐색한다.
 - solve_cube_weighted_astar() 이용

epoch	100/5000		loss=0.00032		1.1 min
epoch	200/5000		loss=0.00090		2.1 min
epoch	300/5000		loss=0.00110		3.2 min
epoch	400/5000		loss=0.00060		4.3 min
epoch	500/5000		loss=0.02107		5.4 min
epoch	600/5000		loss=0.00344		6.5 min
epoch	700/5000		loss=0.00989		7.6 min
epoch	800/5000		loss=0.05680		8.7 min
epoch	900/5000		loss=0.01636		9.8 min
epoch	1000/5000		loss=0.01064		10.9 min
epoch	1100/5000		loss=0.02389		11.9 min
epoch	1200/5000		loss=0.01989		13.1 min
epoch	1300/5000		loss=0.03176		14.2 min
epoch	1400/5000		loss=0.02692		15.3 min
epoch	1500/5000		loss=0.05783		16.4 min
epoch	1600/5000		loss=0.12588		17.5 min
epoch	1700/5000		loss=0.07935		18.6 min
epoch	1800/5000		loss=0.08574		19.7 min
epoch	1900/5000		loss=0.11963		20.9 min
epoch	2000/5000		loss=0.10298		22.0 min
epoch	2100/5000		loss=0.12567		23.1 min
epoch	2200/5000		loss=0.07560		24.2 min
epoch	2300/5000		loss=0.11676		25.3 min
epoch	2400/5000		loss=0.16224		26.4 min
epoch	2500/5000		loss=0.11806		27.5 min
epoch	2600/5000		loss=0.28251		28.6 min
epoch	2700/5000		loss=0.11644		29.7 min
epoch	2800/5000		loss=0.18507		30.8 min
epoch	2900/5000		loss=0.11851		31.9 min
epoch	3000/5000		loss=0.09744		33.0 min

epoch	3100/5000		loss=0.11791		34.1 min
epoch	3200/5000		loss=0.18950		35.2 min
epoch	3300/5000		loss=0.14133		36.3 min
epoch	3400/5000		loss=0.18075		37.4 min
epoch	3500/5000		loss=0.17828		38.5 min
epoch	3600/5000		loss=0.32760		39.6 min
epoch	3700/5000		loss=0.24446		40.7 min
epoch	3800/5000		loss=0.15946		41.8 min
epoch	3900/5000		loss=0.15492		42.9 min
epoch	4000/5000		loss=0.28900		44.0 min
epoch	4100/5000		loss=0.21614		45.1 min
epoch	4200/5000		loss=0.17757		46.2 min
epoch	4300/5000		loss=0.16920		47.3 min
epoch	4400/5000		loss=0.24910		48.4 min
epoch	4500/5000		loss=0.22609		49.5 min
epoch	4600/5000		loss=0.19295		50.6 min
epoch	4700/5000		loss=0.28041		51.7 min
epoch	4800/5000		loss=0.17996		52.8 min
epoch	4900/5000		loss=0.18070		53.9 min
epoch	5000/5000		loss=0.19713		54.9 min

학습 완료

- 5) Weighted A-star의 가중치를 여러 값으로 변경하여 서로 다른 탐색 조건에서 해법을 탐색한다.
- 6) 최종 해법을 반환한다. (탐색 실패시 실패 문구 출력)

IV. 연구 결과

1. 실행 순서

가. Deep Learning Cube Machine

- 1) 신경망을 학습시킨다.
- 2) 최적의 해법을 찾고 싶은 스크램블을 raw_scramble에 넣는다.
- 3) 앞서 학습시킨 신경망을 이용해 A-star 탐색을 한다.
- 4) 도출된 해법을 확인하고 검증한다.

2. 실제 출력 화면

가. Deep Learning Cube Machine

- 1) 회전 엔진 검증

```
device: cuda
회전 엔진 검증 중
회전 엔진 검증 완료
```

- 2) 딥러닝 모델 학습 (epoch = 학습 횟수)

3) 스크램블 검증(37수)

```
스크램블 basic move 수: 37
스크램블 적용 완료

보장된 37수 해법: 37
37수 해법 자체는 정확하게 검증되었습니다.
```

4) Weighted A-star 탐색 및 해법 도출

```
### weight=1.0 | max_nodes=50000 ###
탐색 5000/50000 | g=5 | 10.6s | 472 nodes/s
탐색 10000/50000 | g=7 | 21.5s | 465 nodes/s
탐색 15000/50000 | g=7 | 32.5s | 462 nodes/s
탐색 20000/50000 | g=8 | 43.5s | 460 nodes/s
탐색 25000/50000 | g=5 | 54.1s | 462 nodes/s
탐색 30000/50000 | g=7 | 64.8s | 463 nodes/s
탐색 35000/50000 | g=5 | 75.8s | 462 nodes/s
탐색 40000/50000 | g=7 | 86.9s | 460 nodes/s
탐색 45000/50000 | g=8 | 97.9s | 460 nodes/s
탐색 50000/50000 | g=6 | 108.5s | 461 nodes/s
실패 (108.8초)
```

```
### weight=1.5 | max_nodes=200000 ###
탐색 5000/200000 | g=6 | 10.4s | 482 nodes/s
탐색 10000/200000 | g=7 | 21.2s | 471 nodes/s
탐색 15000/200000 | g=9 | 32.2s | 466 nodes/s
탐색 20000/200000 | g=8 | 43.1s | 464 nodes/s
탐색 25000/200000 | g=9 | 54.2s | 462 nodes/s
탐색 30000/200000 | g=9 | 64.6s | 465 nodes/s
탐색 35000/200000 | g=8 | 75.4s | 464 nodes/s
탐색 40000/200000 | g=9 | 86.5s | 463 nodes/s
탐색 45000/200000 | g=6 | 97.4s | 462 nodes/s
탐색 50000/200000 | g=8 | 108.4s | 461 nodes/s
탐색 55000/200000 | g=6 | 118.9s | 463 nodes/s
탐색 60000/200000 | g=11 | 129.7s | 462 nodes/s
탐색 65000/200000 | g=6 | 140.8s | 462 nodes/s
탐색 70000/200000 | g=8 | 151.8s | 461 nodes/s
탐색 75000/200000 | g=9 | 162.8s | 461 nodes/s
탐색 80000/200000 | g=9 | 173.2s | 462 nodes/s
탐색 85000/200000 | g=10 | 184.1s | 462 nodes/s
탐색 90000/200000 | g=9 | 195.1s | 461 nodes/s
탐색 95000/200000 | g=7 | 206.1s | 461 nodes/s
탐색 100000/200000 | g=11 | 217.1s | 461 nodes/s
탐색 105000/200000 | g=7 | 227.5s | 461 nodes/s
탐색 110000/200000 | g=10 | 238.4s | 461 nodes/s
탐색 115000/200000 | g=10 | 249.5s | 461 nodes/s
탐색 120000/200000 | g=8 | 260.6s | 461 nodes/s
탐색 125000/200000 | g=8 | 271.6s | 460 nodes/s
탐색 130000/200000 | g=10 | 282.1s | 461 nodes/s
탐색 135000/200000 | g=6 | 293.0s | 461 nodes/s
탐색 140000/200000 | g=8 | 304.2s | 460 nodes/s
탐색 145000/200000 | g=9 | 315.2s | 460 nodes/s
탐색 150000/200000 | g=6 | 326.3s | 460 nodes/s
탐색 155000/200000 | g=10 | 336.7s | 460 nodes/s
탐색 160000/200000 | g=10 | 347.6s | 460 nodes/s
탐색 165000/200000 | g=6 | 358.7s | 460 nodes/s
탐색 170000/200000 | g=7 | 369.7s | 460 nodes/s
탐색 175000/200000 | g=10 | 380.6s | 460 nodes/s
탐색 180000/200000 | g=12 | 391.2s | 460 nodes/s
탐색 185000/200000 | g=14 | 402.0s | 460 nodes/s
탐색 190000/200000 | g=12 | 413.0s | 460 nodes/s
탐색 195000/200000 | g=9 | 424.1s | 460 nodes/s
탐색 200000/200000 | g=12 | 435.1s | 460 nodes/s
실패 (436.2초)
```

```
### weight=1.25 | max_nodes=100000 ###
탐색 5000/100000 | g=4 | 10.6s | 471 nodes/s
탐색 10000/100000 | g=5 | 21.6s | 462 nodes/s
탐색 15000/100000 | g=5 | 32.6s | 460 nodes/s
탐색 20000/100000 | g=8 | 43.7s | 458 nodes/s
탐색 25000/100000 | g=7 | 54.4s | 459 nodes/s
탐색 30000/100000 | g=4 | 65.0s | 461 nodes/s
탐색 35000/100000 | g=6 | 76.0s | 460 nodes/s
탐색 40000/100000 | g=5 | 87.0s | 460 nodes/s
탐색 45000/100000 | g=5 | 98.0s | 459 nodes/s
탐색 50000/100000 | g=7 | 108.8s | 460 nodes/s
탐색 55000/100000 | g=10 | 119.4s | 461 nodes/s
탐색 60000/100000 | g=9 | 130.4s | 460 nodes/s
탐색 65000/100000 | g=7 | 141.4s | 460 nodes/s
탐색 70000/100000 | g=7 | 152.5s | 459 nodes/s
탐색 75000/100000 | g=7 | 163.3s | 459 nodes/s
탐색 80000/100000 | g=12 | 173.8s | 460 nodes/s
탐색 85000/100000 | g=9 | 184.9s | 460 nodes/s
탐색 90000/100000 | g=10 | 196.0s | 459 nodes/s
탐색 95000/100000 | g=6 | 207.0s | 459 nodes/s
탐색 100000/100000 | g=8 | 217.9s | 459 nodes/s
실패 (218.6초)
```

```
### weight=2.0 | max_nodes=400000 ###
해법 발견!
길이: 21수
weight: 2.0
시간: 6.3초
해법: R' D' L F U R' U' R F' R F B L' R D D L' R F' L' R'

최종 결과: 21수
해법: R' D' L F U R' U' R F' R F B L' R D D L' R F' L' R'
해법 검증 성공
37수보다 16수 짧음
```

V. 결론 및 제언

1. 결론

- 가. 현재 FMC 세계 신기록 스크램블을 DLCM에 적용시켜 해법을 도출해 본 결과, 세계신기록 해법은 “D2 F' D2 U2 F' L2 D R2 D B2 F L2 R' F' D U'” 인 것에 반해, DLCM의 해법은 “R' D' L F U R' U' R F' R F B L' R D2 L' R F' L' R'”로서 4수가 더 많았다.
- 나. 동일한 스크램블을 현재 최대 성능의 큐브 해법 제시 모듈 Kociemba와 비교해 본 결과, Kociemba의 해법 “B U2 F' B2 R2 D F2 L D B' U B2 U' L2 D L2 U L2 D B2”도 DLCM과 마찬가지로 20수 해법을 찾아냈다.

2. 제언

- 가. 추후에 DLCM의 학습량 epoch를 늘려서 더욱 더 고성능으로 발전시키고 싶다.
- 나. 솔빙 과정을 시각화해서 많은 사람들이 해법을 이해할 수 있도록 하고 싶다.

※참고문헌 : Agostinelli, F., McAleer, S., Shmakov, A., & Baldi, P. (2019). *Solving the Rubik's cube with deep reinforcement learning and search*. Nature Machine Intelligence